Introduction to Haskell Programming

Sorting

S P Suresh

Chennai Mathematical Institute

Haskell @ CMI

Merge sort

- Divide list in two equal parts

Merge sort

- Divide list in two equal parts
- Separately sort left and right half

Merge sort

- Divide list in two equal parts
- Separately sort left and right half
- **Merge** the two sorted halves to get the full list sorted

Merge sort

- Divide list in two equal parts
- Separately sort left and right half
- **Merge** the two sorted halves to get the full list sorted
  - Given two sorted lists `xs` and `ys`, merge into a sorted list `zs`

Merge sort

- Divide list in two equal parts
- Separately sort left and right half
- **Merge** the two sorted halves to get the full list sorted
  - Given two sorted lists `xs` and `ys`, merge into a sorted list `zs`
  - Compare first element of `xs` and `ys`

Merge sort

- Divide list in two equal parts
- Separately sort left and right half
- **Merge** the two sorted halves to get the full list sorted
  - Given two sorted lists `xs` and `ys`, merge into a sorted list `zs`
  - Compare first element of `xs` and `ys`
  - Move it into `zs`

Merge sort

- Divide list in two equal parts
- Separately sort left and right half
- **Merge** the two sorted halves to get the full list sorted
  - Given two sorted lists `xs` and `ys`, merge into a sorted list `zs`
  - Compare first element of `xs` and `ys`
  - Move it into `zs`
  - Repeat until all elements in `xs` and `ys` are processed

Merging two sorted lists – example

- Merging `[22, 32, 43, 78]` and `[13, 57, 63, 91]`

```
merge [22, 32, 43, 78] [13, 57, 63, 91]
= 13: merge [22, 32, 43, 78] [57, 63, 91]
= 13: 22: merge [32, 43, 78] [57, 63, 91]
= 13: 22: 32: merge [43, 78] [57, 63, 91]
= 13: 22: 32: 43: merge [78] [57, 63, 91]
= 13: 22: 32: 43: 57: merge [78] [63, 91]
= 13: 22: 32: 43: 57: 63: merge [78] [91]
= 13: 22: 32: 43: 57: 63: 78: merge [] [91]
= 13: 22: 32: 43: 57: 63: 78: [91]
= [13, 22, 32, 43, 57, 63, 78, 91]
```

Merge sort

- Sort `l !! 0` to `l !! (n/2-1)`

Merge sort

- Sort `l !! 0` to `l !! (n/2-1)`
- Sort `l !! (n/2)` to `l !! (n-1)`

Merge sort

- Sort `l !! 0` to `l !! (n/2-1)`
- Sort `l !! (n/2)` to `l !! (n-1)`
- Merge sorted halves into `l`

Merge sort

- Sort `l !! 0` to `l !! (n/2-1)`
- Sort `l !! (n/2)` to `l !! (n-1)`
- Merge sorted halves into `l`
- How do we sort the halves?

Merge sort

- Sort `l !! 0` to `l !! (n/2-1)`
- Sort `l !! (n/2)` to `l !! (n-1)`
- Merge sorted halves into `l`
- How do we sort the halves?
- Recursively, using the same strategy!

Merge sort – example

- Sorting `[43, 32, 22, 78, 63, 57, 91, 13]`

```
sort [43, 32, 22, 78, 63, 57, 91, 13]
= merge (sort [43, 32, 22, 78]) (sort [63, 57, 91, 13])
= merge (merge (sort [43,32]) (sort [22, 78]))
        (merge (sort [63,57]) (sort [91, 13]))
= merge (merge (merge (sort [43]) (sort [32])) (merge (sort [22]) (sort [78])))
        (merge (merge (sort [63]) (sort [57])) (merge (sort [91]) (sort [13])))
= merge (merge (merge [43] [32]) (merge [22] [78]))
        (merge (merge [63] [57])  (merge [91] [13]))
= merge (merge [32, 43] [22, 78]) (merge [57, 63] [13, 91])
= merge [22, 32, 43, 78] [13, 57, 63, 91]
= [13, 22, 32, 43, 57, 63, 78, 91]
```

Merge sort: merge

- Merging two sorted lists:

```
merge :: [Int] → [Int] → [Int]
merge []     ys     = ys
merge xs     []     = xs
merge (x:xs) (y:ys)
    | x ⩽ y         = x: merge xs (y:ys)
    | otherwise     = y: merge (x:xs) ys
```

Merge sort: merge

- Merging two sorted lists:

```
merge :: [Int] → [Int] → [Int]
merge []     ys     = ys
merge xs     []     = xs
merge (x:xs) (y:ys)
    | x ⩽ y         = x: merge xs (y:ys)
    | otherwise     = y: merge (x:xs) ys
```

- Each comparison adds at least one element to the output list

Merge sort: merge

- Merging two sorted lists:

```
merge :: [Int] → [Int] → [Int]
merge []     ys     = ys
merge xs     []     = xs
merge (x:xs) (y:ys)
    | x ⩽ y         = x: merge xs (y:ys)
    | otherwise     = y: merge (x:xs) ys
```

- Each comparison adds at least one element to the output list
- Number of steps in `merge xs ys` is $O(n)$

Merge sort: merge

- Merging two sorted lists:

```
merge :: [Int] → [Int] → [Int]
merge []     ys     = ys
merge xs     []     = xs
merge (x:xs) (y:ys)
    | x ⩽ y         = x: merge xs (y:ys)
    | otherwise     = y: merge (x:xs) ys
```

- Each comparison adds at least one element to the output list
- Number of steps in `merge xs ys` is $O(n)$
  - n is `length xs + length ys`

Merge sort

- Sorting a list:

```
sort :: [Int] → [Int]
sort []  = []
sort [x] = [x]
sort xs  = merge (sort front) (sort back) where
    n             = (length xs) `div` 2
    (front, back) = splitAt n xs
```

Merge sort: analysis

- $T(n)$: time taken by `sort` on input of length n

Merge sort: analysis

- $T(n)$: time taken by `sort` on input of length n
- Assume, for simplicity, that n is a power of 2

Merge sort: analysis

- $T(n)$: time taken by `sort` on input of length n
- Assume, for simplicity, that n is a power of 2
- Then the lengths of `front` and `back` are $\frac{n}{2}$

Merge sort: analysis

- $T(n)$: time taken by `sort` on input of length n
- Assume, for simplicity, that n is a power of 2
- Then the lengths of `front` and `back` are $\frac{n}{2}$
- There are two recursive sorts

Merge sort: analysis

- $T(n)$: time taken by `sort` on input of length n
- Assume, for simplicity, that n is a power of 2
- Then the lengths of `front` and `back` are $\frac{n}{2}$
- There are two recursive sorts
- Overall time taken by **`length`**, **`splitAt`** and `merge` is $O(n)$

Merge sort: analysis

- $T(n)$: time taken by `sort` on input of length n
- Assume, for simplicity, that n is a power of 2
- Then the lengths of `front` and `back` are $\frac{n}{2}$
- There are two recursive sorts
- Overall time taken by **`length`**, **`splitAt`** and `merge` is $O(n)$
- Let us assume it is cn, for some constant c

Merge sort: analysis

- $T(n)$: time taken by `sort` on input of length n
- Assume, for simplicity, that n is a power of 2
- Then the lengths of `front` and `back` are $\frac{n}{2}$
- There are two recursive sorts
- Overall time taken by **`length`**, **`splitAt`** and `merge` is $O(n)$
- Let us assume it is cn, for some constant c
- **Recurrence**: $T(n) = 2T(n/2) + cn$

Merge sort: analysis

- **Recurrence**: $T(n) = 2T(n/2) + cn$

$$
\begin{aligned}
T(1) &= 1 \\
T(n) &= 2T(n/2) + cn \\
&= 2(2T(n/4) + cn/2) + cn \\
&= 2^2T(n/2^2) + 2cn \\
&= 2^2(2T(n/2^3) + cn/2^2) + 2cn \\
&= 2^3T(n/2^3) + 3cn \\
&= \ldots \\
&= 2^jT(n/2^j) + jcn
\end{aligned}
$$

Merge sort: analysis

- **Recurrence**: $T(n) = 2T(n/2) + cn$

Merge sort: analysis

- **Recurrence**: $T(n) = 2T(n/2) + cn$
- Expanding ...

$$
\begin{aligned}
T(1) &= 1 \\
T(n) &= 2T(n/2) + cn \\
&= \ldots \\
&= 2^j T(n/2^j) + jcn
\end{aligned}
$$

Merge sort: analysis

- **Recurrence**: $T(n) = 2T(n/2) + cn$
- Expanding ...

$$
\begin{aligned}
T(1) &= 1 \\
T(n) &= 2T(n/2) + cn \\
&= \ldots \\
&= 2^j T(n/2^j) + jcn
\end{aligned}
$$

- When $j = \log n$, $2^j = n$ and $n/2^j = 1$

Merge sort: analysis

- **Recurrence**: $T(n) = 2T(n/2) + cn$
- Expanding ...

$$
\begin{aligned}
T(1) &= 1 \\
T(n) &= 2T(n/2) + cn \\
&= \ldots \\
&= 2^j T(n/2^j) + jcn
\end{aligned}
$$

- When $j = \log n$, $2^j = n$ and $n/2^j = 1$
- Thus $T(n) = 2^{\log n} T(1) + cn \log n = n + cn \log n = O(n \log n)$

Towards an improved merge sort

- Recall the computation of `sort [43, 32, 22, 78, 63, 57, 91, 13]`

```
sort [43, 32, 22, 78, 63, 57, 91, 13]
= merge (sort [43, 32, 22, 78]) (sort [63, 57, 91, 13])
= ...
= merge (merge (merge [43] [32]) (merge [22] [78]))
        (merge (merge [63] [57])  (merge [91] [13]))
= merge (merge [32, 43] [22, 78]) (merge [57, 63] [13, 91])
= merge [22, 32, 43, 78] [13, 57, 63, 91]
= [13, 22, 32, 43, 57, 63, 78, 91]
```

Towards an improved merge sort

- Recall the computation of `sort [43, 32, 22, 78, 63, 57, 91, 13]`

```
sort [43, 32, 22, 78, 63, 57, 91, 13]
= merge (sort [43, 32, 22, 78]) (sort [63, 57, 91, 13])
= ...
= merge (merge (merge [43] [32]) (merge [22] [78]))
        (merge (merge [63] [57])  (merge [91] [13]))
= merge (merge [32, 43] [22, 78]) (merge [57, 63] [13, 91])
= merge [22, 32, 43, 78] [13, 57, 63, 91]
= [13, 22, 32, 43, 57, 63, 78, 91]
```

- The real computation consists in repeatedly merging, starting from singleton lists

Towards an improved merge sort

- Recall the computation of `sort [43, 32, 22, 78, 63, 57, 91, 13]`

```
sort [43, 32, 22, 78, 63, 57, 91, 13]
= merge (sort [43, 32, 22, 78]) (sort [63, 57, 91, 13])
= ...
= merge (merge (merge [43] [32]) (merge [22] [78]))
        (merge (merge [63] [57])  (merge [91] [13]))
= merge (merge [32, 43] [22, 78]) (merge [57, 63] [13, 91])
= merge [22, 32, 43, 78] [13, 57, 63, 91]
= [13, 22, 32, 43, 57, 63, 78, 91]
```

- The real computation consists in repeatedly merging, starting from singleton lists
- We can create a list of singletons from the original list ...

Towards an improved merge sort

- Recall the computation of `sort [43, 32, 22, 78, 63, 57, 91, 13]`

```
sort [43, 32, 22, 78, 63, 57, 91, 13]
= merge (sort [43, 32, 22, 78]) (sort [63, 57, 91, 13])
= ...
= merge (merge (merge [43] [32]) (merge [22] [78]))
        (merge (merge [63] [57])  (merge [91] [13]))
= merge (merge [32, 43] [22, 78]) (merge [57, 63] [13, 91])
= merge [22, 32, 43, 78] [13, 57, 63, 91]
= [13, 22, 32, 43, 57, 63, 78, 91]
```

- The real computation consists in repeatedly merging, starting from singleton lists
- We can create a list of singletons from the original list ...
- ...and repeatedly merge adjacent pairs till we are left with a single list!

Improved merge sort

```
singles                 = map (:[])
-- singles [x1,...,xn]  = [[x1], ..., [xn]]

mergePairs []           = []
mergePairs [l]          = [l]
mergePairs (l1:l2:ls)   = merge l1 l2: mergePairs ls

mergeAll []             = []
mergeAll [l]            = l
mergeAll xs             = mergeAll (mergePairs xs)

sort                    = mergeAll . singles
```

Analysis and further improvements

- `singles` takes $O(n)$ time to transform the list `l = [x1, ... , xn]` to `[[x1], ... , [xn]]`

Analysis and further improvements

- `singles` takes $O(n)$ time to transform the list `l = [x1, ... , xn]` to `[[x1], ... , [xn]]`
- `mergePairs` on a list `[l1, ... , lm]` takes $O(k_1 + \cdots + k_m)$ number of steps

Analysis and further improvements

- `singles` takes $O(n)$ time to transform the list `l = [x1, ... , xn]` to `[[x1], ... , [xn]]`
- `mergePairs` on a list `[l1, ... , lm]` takes $O(k_1 + \cdots + k_m)$ number of steps
  - k_i is **`length`** `li`, for $1 \leqslant i \leqslant m$

Analysis and further improvements

- `singles` takes $O(n)$ time to transform the list `l = [x1, ... , xn]` to `[[x1], ... , [xn]]`
- `mergePairs` on a list `[l1, ... , lm]` takes $O(k_1 + \cdots + k_m)$ number of steps
  - k_i is `length li`, for $1 \leqslant i \leqslant m$
- `mergePairs` reduces a list with m lists to one with $\lceil m/2 \rceil$ lists

Analysis and further improvements

- `singles` takes $O(n)$ time to transform the list `l = [x1, ..., xn]` to `[[x1], ..., [xn]]`
- `mergePairs` on a list `[l1, ..., lm]` takes $O(k_1 + \cdots + k_m)$ number of steps
  - k_i is `length li`, for $1 \leqslant i \leqslant m$
- `mergePairs` reduces a list with m lists to one with $\lceil m/2 \rceil$ lists
- After $O(\log n)$ recursive calls of `mergeAll`

Analysis and further improvements

- `singles` takes $O(n)$ time to transform the list `l = [x1, ..., xn]` to `[[x1], ..., [xn]]`
- `mergePairs` on a list `[l1, ..., lm]` takes $O(k_1 + \cdots + k_m)$ number of steps
  - k_i is `length li`, for $1 \leqslant i \leqslant m$
- `mergePairs` reduces a list with m lists to one with $\lceil m/2 \rceil$ lists
- After $O(\log n)$ recursive calls of `mergeAll`
  - the list `[[x1], ..., [xn]]` is reduced to a singleton list `[l']`

Analysis and further improvements

- `singles` takes $O(n)$ time to transform the list `l = [x1, ..., xn]` to `[[x1], ..., [xn]]`
- `mergePairs` on a list `[l1, ..., lm]` takes $O(k_1 + \cdots + k_m)$ number of steps
  - k_i is `length li`, for $1 \leqslant i \leqslant m$
- `mergePairs` reduces a list with m lists to one with $\lceil m/2 \rceil$ lists
- After $O(\log n)$ recursive calls of `mergeAll`
  - the list `[[x1], ..., [xn]]` is reduced to a singleton list `[l']`
  - `l'` is `l` in sorted order

Analysis and further improvements

- `singles` takes $O(n)$ time to transform the list `l = [x1, ..., xn]` to `[[x1], ..., [xn]]`
- `mergePairs` on a list `[l1, ..., lm]` takes $O(k_1 + \cdots + k_m)$ number of steps
  - k_i is `length li`, for $1 \leqslant i \leqslant m$
- `mergePairs` reduces a list with m lists to one with $\lceil m/2 \rceil$ lists
- After $O(\log n)$ recursive calls of `mergeAll`
  - the list `[[x1], ..., [xn]]` is reduced to a singleton list `[l']`
  - `l'` is `l` in sorted order
- Thus the overall time taken is $O(n \log n)$

Analysis and further improvements

- `singles` takes $O(n)$ time to transform the list `l = [x1, ..., xn]` to `[[x1], ..., [xn]]`
- `mergePairs` on a list `[l1, ..., lm]` takes $O(k_1 + \cdots + k_m)$ number of steps
  - k_i is `length li`, for $1 \leqslant i \leqslant m$
- `mergePairs` reduces a list with m lists to one with $\lceil m/2 \rceil$ lists
- After $O(\log n)$ recursive calls of `mergeAll`
  - the list `[[x1], ..., [xn]]` is reduced to a singleton list `[l']`
  - `l'` is `l` in sorted order
- Thus the overall time taken is $O(n \log n)$
- Instead of starting with singleton lists, one can start with **up runs** and the reverse of **down runs**

Analysis and further improvements

- `singles` takes $O(n)$ time to transform the list `l = [x1, ..., xn]` to `[[x1], ..., [xn]]`
- `mergePairs` on a list `[l1, ..., lm]` takes $O(k_1 + \cdots + k_m)$ number of steps
  - k_i is `length li`, for $1 \leqslant i \leqslant m$
- `mergePairs` reduces a list with m lists to one with $\lceil m/2 \rceil$ lists
- After $O(\log n)$ recursive calls of `mergeAll`
  - the list `[[x1], ..., [xn]]` is reduced to a singleton list `[l']`
  - `l'` is `l` in sorted order
- Thus the overall time taken is $O(n \log n)$
- Instead of starting with singleton lists, one can start with **up runs** and the reverse of **down runs**
  - An up run is a maximal contiguous segment sorted in ascending order

Analysis and further improvements

- `singles` takes $O(n)$ time to transform the list `l = [x1, ..., xn]` to `[[x1], ..., [xn]]`
- `mergePairs` on a list `[l1, ..., lm]` takes $O(k_1 + \cdots + k_m)$ number of steps
  - k_i is `length li`, for $1 \leqslant i \leqslant m$
- `mergePairs` reduces a list with m lists to one with $\lceil m/2 \rceil$ lists
- After $O(\log n)$ recursive calls of `mergeAll`
  - the list `[[x1], ..., [xn]]` is reduced to a singleton list `[l']`
  - `l'` is `l` in sorted order
- Thus the overall time taken is $O(n \log n)$
- Instead of starting with singleton lists, one can start with **up runs** and the reverse of **down runs**
  - An up run is a maximal contiguous segment sorted in ascending order
  - A down run is a maximal contiguous segment sorted in descending order

Avoiding merge

- Merge is needed because some elements in `front` are greater than some elements in `back`

Avoiding merge

- Merge is needed because some elements in `front` are greater than some elements in `back`
- Can we ensure that everything in `front` is smaller than everything in `back`?

Avoiding merge

- Merge is needed because some elements in `front` are greater than some elements in `back`
- Can we ensure that everything in `front` is smaller than everything in `back`?
- Suppose the median value in list is `m`

Avoiding merge

- Merge is needed because some elements in `front` are greater than some elements in `back`
- Can we ensure that everything in `front` is smaller than everything in `back`?
- Suppose the median value in list is `m`
- Move all values $\leqslant$ `m` to `front`

Avoiding merge

- Merge is needed because some elements in `front` are greater than some elements in `back`
- Can we ensure that everything in `front` is smaller than everything in `back`?
- Suppose the median value in list is `m`
- Move all values $\leqslant$ `m` to `front`
- `back` has values `> m`

Avoiding merge

- Merge is needed because some elements in `front` are greater than some elements in `back`
- Can we ensure that everything in `front` is smaller than everything in `back`?
- Suppose the median value in list is `m`
- Move all values $\leqslant$ `m` to `front`
- `back` has values `> m`
- Recursively sort `front` and `back`

Avoiding merge

- Merge is needed because some elements in `front` are greater than some elements in `back`
- Can we ensure that everything in `front` is smaller than everything in `back`?
- Suppose the median value in list is `m`
- Move all values $\leqslant$ `m` to `front`
- `back` has values `> m`
- Recursively sort `front` and `back`
- List is now sorted! No need to merge

Avoiding merge

- How do we find the median?

Avoiding merge

- How do we find the median?
- Sort and pick up middle element

Avoiding merge

- How do we find the median?
- Sort and pick up middle element
- But our aim is to sort!

Avoiding merge

- How do we find the median?
- Sort and pick up middle element
- But our aim is to sort!
- Instead, pick up some value in list – **pivot**

Avoiding merge

- How do we find the median?
- Sort and pick up middle element
- But our aim is to sort!
- Instead, pick up some value in list – **pivot**
- Split list with respect to the pivot

Avoiding merge

- How do we find the median?
- Sort and pick up middle element
- But our aim is to sort!
- Instead, pick up some value in list – **pivot**
- Split list with respect to the pivot
- Usually we pick the first element as pivot

Quicksort

```
sort :: [Int] → [Int]
sort []     = []
sort (x:xs) = sort front ++ [pivot] ++ sort back where
    pivot   = x
    front   = [y ← xs, y ⩽ x]
    back    = [y ← xs, y > x]
```

Quicksort: analysis

- **Worst case**: `pivot` is maximum or minimum

Quicksort: analysis

- **Worst case**: `pivot` is maximum or minimum
- One partition is empty while the other is of size $n - 1$

Quicksort: analysis

- **Worst case**: `pivot` is maximum or minimum
- One partition is empty while the other is of size $n - 1$
- Partitioning takes time $O(n)$, say cn

Quicksort: analysis

- **Worst case**: `pivot` is maximum or minimum
- One partition is empty while the other is of size $n - 1$
- Partitioning takes time $O(n)$, say cn
- $T(n) = T(n-1) + cn$

Quicksort: analysis

- **Worst case**: `pivot` is maximum or minimum
- One partition is empty while the other is of size $n - 1$
- Partitioning takes time $O(n)$, say cn
- $T(n) = T(n-1) + cn$
- Expanding, $T(n) = O(n^2)$

Quicksort: analysis

- **Worst case**: `pivot` is maximum or minimum
- One partition is empty while the other is of size $n-1$
- Partitioning takes time $O(n)$, say cn
- $T(n) = T(n-1) + cn$
- Expanding, $T(n) = O(n^2)$
- But **average case complexity** is $O(n \log n)$

Quicksort: analysis

- **Worst case**: `pivot` is maximum or minimum
- One partition is empty while the other is of size $n-1$
- Partitioning takes time $O(n)$, say cn
- $T(n) = T(n-1) + cn$
- Expanding, $T(n) = O(n^2)$
- But **average case complexity** is $O(n \log n)$
- Quicksort is one of the few examples amenable to average case analysis

Average case analysis

- Assume input is a permutation of $1, 2, \ldots, n$

Average case analysis

- Assume input is a permutation of $1, 2, \ldots, n$
- Actual values not important

Average case analysis

- Assume input is a permutation of $1, 2, \ldots, n$
- Actual values not important
- Only relative order matters

Average case analysis

- Assume input is a permutation of $1, 2, \ldots, n$
- Actual values not important
- Only relative order matters
- Each permutation is equally likely as input (uniform probability)

Average case analysis

- Assume input is a permutation of $1, 2, \ldots, n$
- Actual values not important
- Only relative order matters
- Each permutation is equally likely as input (uniform probability)
- Calculate running time across all inputs

Average case analysis

- Assume input is a permutation of $1, 2, \ldots, n$
- Actual values not important
- Only relative order matters
- Each permutation is equally likely as input (uniform probability)
- Calculate running time across all inputs
- Expected running time can be shown to be $O(n \log n)$